FINAL YEAR PROJECT – COMPLETE SETUP GUIDE

From Zero to

Autonomous Robot

Navigation

Windows + MobaXterm | Ubuntu 22.04 | Raspberry Pi 4

QBot 2 | Kinect | ROS 2 Humble | SLAM | Nav2

Student	Baanujan | E/20/030
Supervisor	Dr. D.H.S. Maithripala
Hardware	Raspberry Pi 4 + Quanser QBot 2 + Xbox Kinect
Software	Ubuntu 22.04 LTS + ROS 2 Humble + RTAB-Map + Nav2
Development PC	Windows with MobaXterm for SSH and X11 forwarding
Goal	SLAM map building + autonomous navigation

Follow every step in order. Do not skip. Come back here whenever you get stuck.

Contents

How to Use This Guide

This guide takes you from a blank SD card all the way to a robot navigating autonomously. It is written for a beginner. Every command is given clearly, and each stage explains what you are doing.

Symbol	Meaning
NOTE	Important information you should read.
WARNING	If this is skipped, an error may occur.
TIP	Helpful advice for beginners.
WINDOWS	Something you do on your Windows laptop, not on the Raspberry Pi.

NOTE

Every command that starts with `$` means you type everything after the `$` sign. Do not type the `$` itself.

WINDOWS

Commands you run on your Windows laptop are marked with WINDOWS. Everything else is typed in the MobaXterm SSH terminal connected to the Raspberry Pi.

PHASE 0
Set Up Your Windows Laptop

Before touching the Raspberry Pi, set up two things on your Windows laptop.

Download Raspberry Pi Imager

WINDOWS

Open your browser and go to:
`https://www.raspberrypi.com/software/`
Click **Download for Windows** and install it. This tool is used to flash Ubuntu onto the SD card.

Check MobaXterm X11 Settings

WINDOWS

Open MobaXterm. Click **Settings** at the top menu. Click **Configuration**. Then click the **X11** tab. Make sure **X11 forwarding** is enabled. Click **OK**.

This lets RViz2, the robot visualisation tool, appear on your Windows screen.

NOTE

MobaXterm already has X11 built in. You do not need to install anything extra. This is why MobaXterm is useful for robotics work.

PHASE 1
Flash Ubuntu 22.04 onto Raspberry Pi 4

This phase completely wipes the current Debian system and installs Ubuntu 22.04, which is the correct operating system for ROS 2 Humble.

WARNING

This will erase everything on the SD card, including the current Debian system. Save anything important before continuing.

STEP 1 – Insert the MicroSD Card

Insert the MicroSD card into your Windows laptop using a card reader.

STEP 2 – Open Raspberry Pi Imager

Open the Raspberry Pi Imager software on Windows.

STEP 3 – Choose the Operating System

Click **Choose OS**. Then select:
Other general-purpose OS → Ubuntu → Ubuntu Server 22.04 LTS 64-bit
Choose the 64-bit Server version, not Desktop. Server is lighter and better for the robot.

STEP 4 – Choose Your Storage

Click **Choose Storage** and select your MicroSD card. Be careful to select the correct drive.

STEP 5 – Open Advanced Settings

Click the gear icon in Raspberry Pi Imager and fill these settings:

```
Hostname:       raspberrypi
Enable SSH:     Yes
Username:       ubuntu
Password:       Choose a password you will remember
WiFi SSID:      Your lab WiFi name, if using WiFi
WiFi Password:  Your lab WiFi password
WiFi Country:   LK
```

TIP

Setting these values in advance means the Raspberry Pi will be ready for SSH after the first boot. No monitor or keyboard is needed.

STEP 6 – Write the Image

Click **Write**. Confirm the erase message. Wait until writing is complete. Do not remove the card while it is writing.

STEP 7 – Insert SD Card into Raspberry Pi

When Raspberry Pi Imager says **Write Successful**, remove the card and insert it into the Raspberry Pi 4.

STEP 8 – Power on the Raspberry Pi

Connect the Ethernet cable, or use the WiFi settings you configured. Then connect USB-C power. Wait around 3 minutes for the first boot.

Connect to Raspberry Pi from MobaXterm

WINDOWS

Open MobaXterm. Click **Session**. Select **SSH**. Fill in:

```
Remote host: raspberrypi.local
Username:    ubuntu
Port:        22
```

Click **OK**.

If `raspberrypi.local` does not work, find the Raspberry Pi IP address from the router device list and use that IP address instead.

```
# Type yes when asked about the fingerprint.
# Enter your password when asked.
# You should see:

ubuntu@raspberrypi:~$

# You are now inside the Raspberry Pi.
```

NOTE

From now on, commands are typed in the MobaXterm SSH window. You are controlling the Raspberry Pi from your Windows laptop.

PHASE 2

Update Ubuntu and Install Essential Tools

Always update the system before installing anything. This ensures the Raspberry Pi has the latest package lists and compatible software.

```
# Update package list
sudo apt update

# Upgrade installed packages
sudo apt upgrade -y

# Install essential tools
sudo apt install -y curl wget git build-essential python3-pip

# Reboot the Raspberry Pi
sudo reboot
```

WINDOWS

After reboot, wait around one minute, then reconnect using MobaXterm.

NOTE

`sudo` means run as administrator. It will ask for your password sometimes.

PHASE 3

Install ROS 2 Humble

ROS 2 is the framework that connects all robot software together. The Kinect, wheels, SLAM, and navigation all communicate through ROS 2.

Step 1 – Set Locale

```
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Step 2 – Add ROS 2 Repository

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install -y curl

sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Step 3 – Install ROS 2 Humble

```
sudo apt update
sudo apt install -y ros-humble-ros-base
sudo apt install -y python3-colcon-common-extensions python3-rosdep python3-
    argcomplete
```

WARNING

This download can be large. Make sure the internet connection is stable. If the installation fails halfway, run the same command again.

Step 4 – Configure ROS 2 Environment

```
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

Step 5 – Test ROS 2

Open two SSH sessions to the Raspberry Pi in MobaXterm.

```
# TAB 1
ros2 run demo_nodes_cpp talker

# TAB 2
ros2 run demo_nodes_cpp listener

# TAB 2 should print:
# [INFO] I heard: Hello World: 1
# [INFO] I heard: Hello World: 2

# Press Ctrl+C in both tabs to stop.
```

NOTE

If you see Hello World messages, ROS 2 is installed and working correctly.

PHASE 4

Install Kobuki Driver for QBot 2 Wheels

The QBot 2 uses a Kobuki mobile base. This driver lets ROS 2 control the left and right wheels and read odometry data.

Create ROS 2 Workspace

```
mkdir -p ~/fyp_ws/src
cd ~/fyp_ws/src
```

Download Kobuki Packages

```
git clone https://github.com/kobuki-base/kobuki_ros.git
git clone https://github.com/kobuki-base/kobuki_ros_interfaces.git
git clone https://github.com/kobuki-base/kobuki_core.git
git clone https://github.com/kobuki-base/velocity_smoother.git
git clone https://github.com/stonier/ecl_tools.git
```

Install Dependencies

```
cd ~/fyp_ws

# Initialise rosdep. Run this only once.
sudo rosdep init
rosdep update

# Install dependencies
rosdep install --from-paths src --ignore-src -r -y
```

Build the Workspace

```
cd ~/fyp_ws
colcon build --symlink-install

# Add workspace to shell
echo 'source ~/fyp_ws/install/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

WARNING

If `colcon build` gives errors, run the `rosdep install` command again and rebuild.

Connect and Test QBot 2

```
# Connect QBot 2 to Raspberry Pi using USB.
# Check whether it is detected:
ls /dev/ttyUSB*

# It should show:
# /dev/ttyUSB0

# Give permission to use the USB port:
sudo chmod 666 /dev/ttyUSB0

# Make permission permanent:
sudo usermod -aG dialout ubuntu
sudo reboot
```

After reboot, reconnect MobaXterm and continue:

```
# Launch Kobuki driver:
ros2 launch kobuki_ros kobuki_node-launch.py

# Open a second MobaXterm tab and check:
ros2 topic list

# Look for:
# /odom
# /cmd_vel
# /joint_states
```

NOTE

If `/odom` appears in the topic list, the QBot 2 wheels are connected and working.

PHASE 5

Install Kinect Driver

The Kinect sensor gives the robot depth information. It can measure how far objects are from the robot.

Identify Your Kinect Version

	Kinect v1	**Kinect v2**
How to identify	Xbox 360 Kinect or smaller body	Xbox One Kinect or wider body
USB connection	USB 2.0	USB 3.0 only
Driver	freenect + openni2	libfreenect2 + iai_kinect2
Recommended section	Section A	Section B

Section A – Kinect v1

Install freenect library

```
sudo apt install -y libfreenect-dev freenect

# Test whether Kinect is detected:
lsusb

# Look for:
# Microsoft Corp. Xbox NUI
# or PrimeSense
```

Install OpenNI2 ROS 2 package

```
sudo apt install -y ros-humble-openni2-camera ros-humble-openni2-launch
```

Set USB Permissions

```
sudo nano /etc/udev/rules.d/66-kinect.rules
```

Paste the following lines:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="045e", ATTR{idProduct}=="02ae", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="045e", ATTR{idProduct}=="02bf", MODE="0666"
```

TIP

To save in nano, press **Ctrl+O**, then **Enter**, then **Ctrl+X**.

```
sudo udevadm control --reload-rules && sudo udevadm trigger
sudo reboot
```

After reboot, test the Kinect:

```
# Launch Kinect ROS 2 node:
ros2 launch openni2_launch openni2.launch.xml

# Open a second tab and check topics:
ros2 topic list

# You should see:
# /camera/color/image_raw
# /camera/depth/image_raw
# /camera/depth/camera_info

# Check publishing rate:
ros2 topic hz /camera/depth/image_raw

# Expected rate:
# average rate: 30.000
```

NOTE

If the Kinect publishes depth data at around 30 Hz, it is working correctly.

Section B – Kinect v2

WARNING

Only follow this section if your sensor is Kinect v2. If it is Kinect v1, skip this part.

```
sudo apt install -y libfreenect2-dev

cd ~/fyp_ws/src
git clone https://github.com/paul-shuvo/iai_kinect2_humble.git

cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

```
# Launch:
ros2 launch kinect2_bridge kinect2_bridge.launch.xml

# Check topics:
ros2 topic list

# Look for:
# /kinect2/sd/image_depth_rect
```

PHASE 6

Convert Kinect Depth to Laser Scan

SLAM and navigation algorithms normally expect laser scan data. Since Kinect is a depth camera, the depth image is converted into a 2D laser scan.

Install the Package

```
sudo apt install -y ros-humble-depthimage-to-laserscan
```

Create a Launch File

```
mkdir -p ~/fyp_ws/src/qbot2_bringup/launch
nano ~/fyp_ws/src/qbot2_bringup/launch/depth_to_scan.launch.py
```

Paste the following code:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='depthimage_to_laserscan',
            executable='depthimage_to_laserscan_node',
            name='depthimage_to_laserscan',
            remappings=[
                ('depth', '/camera/depth/image_raw'),
                ('depth_camera_info', '/camera/depth/camera_info'),
                ('scan', '/scan'),
            ],
            parameters=[{
                'scan_height': 1,
                'range_min': 0.45,
                'range_max': 4.0,
                'output_frame': 'camera_depth_frame',
            }]
        )
    ])
```

Create Package Files

```
nano ~/fyp_ws/src/qbot2_bringup/package.xml
```

Paste:

```
<?xml version="1.0"?>
<package format="3">
  <name>qbot2_bringup</name>
  <version>0.0.1</version>
  <description>QBot 2 bringup launch files</description>
  <maintainer email="test@test.com">ubuntu</maintainer>
  <license>Apache-2.0</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <exec_depend>depthimage_to_laserscan</exec_depend>
</package>
```

Create `CMakeLists.txt`:

```
nano ~/fyp_ws/src/qbot2_bringup/CMakeLists.txt
```

Paste:

```
cmake_minimum_required(VERSION 3.8)
project(qbot2_bringup)

find_package(ament_cmake REQUIRED)

install(DIRECTORY launch DESTINATION share/${PROJECT_NAME}/)

ament_package()
```

Build the workspace:

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

Test the Conversion

Use three MobaXterm tabs:

```
# TAB 1 -- Kinect
ros2 launch openni2_launch openni2.launch.xml

# TAB 2 -- Depth to LaserScan
ros2 launch qbot2_bringup depth_to_scan.launch.py
```

```
# TAB 3 -- Check /scan topic
ros2 topic echo /scan

# You should see scan range data.
```

TIP

The `/scan` topic is the bridge between the Kinect and the SLAM algorithm. Once this appears, the Kinect is acting like a laser sensor.

PHASE 7
Create Robot Description using URDF

ROS 2 needs to know the physical structure of the robot. This is described using a URDF file.

WARNING

Measure the actual height of the Kinect from the floor before doing this step. For example, if the Kinect height is 28 cm, use `0.28`.

```
mkdir -p ~/fyp_ws/src/qbot2_description/urdf
nano ~/fyp_ws/src/qbot2_description/urdf/qbot2.urdf
```

Paste the following URDF. Change `0.28` to the actual Kinect height:

```
<?xml version="1.0"?>
<robot name="qbot2">

  <link name="base_footprint"/>

  <joint name="base_joint" type="fixed">
    <parent link="base_footprint"/>
    <child link="base_link"/>
    <origin xyz="0 0 0.0102" rpy="0 0 0"/>
  </joint>

  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.10" radius="0.178"/>
      </geometry>
    </visual>
  </link>

  <!-- Change 0.28 to the actual Kinect height in metres -->
  <joint name="camera_joint" type="fixed">
    <parent link="base_link"/>
    <child link="camera_link"/>
    <origin xyz="0.05 0.0 0.28" rpy="0 0 0"/>
  </joint>

  <link name="camera_link"/>

  <joint name="camera_depth_joint" type="fixed">
    <parent link="camera_link"/>
    <child link="camera_depth_frame"/>
```

```
    <origin xyz="0 0.025 0" rpy="-1.5708 0 -1.5708"/>
  </joint>

  <link name="camera_depth_frame"/>

</robot>
```

Create package files:

```
nano ~/fyp_ws/src/qbot2_description/package.xml
```

Paste:

```
<?xml version="1.0"?>
<package format="3">
  <name>qbot2_description</name>
  <version>0.0.1</version>
  <description>QBot 2 URDF description</description>
  <maintainer email="test@test.com">ubuntu</maintainer>
  <license>Apache-2.0</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>joint_state_publisher</exec_depend>
</package>
```

Create `CMakeLists.txt`:

```
nano ~/fyp_ws/src/qbot2_description/CMakeLists.txt
```

Paste:

```
cmake_minimum_required(VERSION 3.8)
project(qbot2_description)

find_package(ament_cmake REQUIRED)

install(DIRECTORY urdf DESTINATION share/${PROJECT_NAME}/)

ament_package()
```

Build:

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

PHASE 8

Set Up Robot State Publisher

The robot state publisher reads the URDF and broadcasts the coordinate frames of the robot. SLAM and Nav2 need this to understand where the robot base and camera are located.

```
sudo apt install -y ros-humble-robot-state-publisher ros-humble-joint-state-
    publisher
```

Create the launch file:

```
nano ~/fyp_ws/src/qbot2_bringup/launch/robot_state.launch.py
```

Paste:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    urdf_file = os.path.join(
        get_package_share_directory('qbot2_description'),
        'urdf',
        'qbot2.urdf'
    )

    with open(urdf_file, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}]
        )
    ])
```

Build again:

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

PHASE 9

Install SLAM and Build the Map

SLAM means **Simultaneous Localisation and Mapping**. The robot moves around the room, builds a map, and estimates its own position at the same time.

Install RTAB-Map

```
sudo apt install -y ros-humble-rtabmap-ros
```

NOTE

RTAB-Map is suitable for RGB-D cameras like the Kinect. It can use both RGB and depth information for mapping.

Launch Everything for Mapping

Use four MobaXterm SSH tabs.

Tab 1 – Kobuki Wheels

```
ros2 launch kobuki_ros kobuki_node-launch.py
```

Tab 2 – Kinect

```
ros2 launch openni2_launch openni2.launch.xml
```

Tab 3 – Robot State Publisher

```
ros2 launch qbot2_bringup robot_state.launch.py
```

Tab 4 – RTAB-Map SLAM

```
ros2 launch rtabmap_launch rtabmap.launch.py \
  rgb_topic:=/camera/color/image_raw \
  depth_topic:=/camera/depth/image_raw \
  camera_info_topic:=/camera/color/camera_info \
  frame_id:=base_footprint \
  approx_sync:=true \
  args:='-d' \
```

```
  rviz:=false
```

NOTE

The `-d` flag deletes the previous RTAB-Map database and starts a fresh map. Remove it when you want to continue using an existing database.

Drive the Robot to Build the Map

Open a fifth tab:

```
sudo apt install -y ros-humble-teleop-twist-keyboard
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Key	Action	Speed Tip
i	Move forward	Start slowly.
,	Move backward	Use carefully.
j	Turn left	Use smooth turns.
l	Turn right	Avoid sharp spins.
k	Stop immediately	Press this if unsure.
u	Forward left curve	Useful for smooth mapping.
o	Forward right curve	Useful for smooth mapping.
q	Increase speed	Use carefully.
z	Decrease speed	Recommended for mapping.

TIP

Drive slowly around the whole lab. Sharp turns and fast motion can make the map messy.

Watch the Map in RViz2

WINDOWS

Open another MobaXterm tab with X11 forwarding and run:

```
ssh -X ubuntu@raspberrypi.local
rviz2
```

In RViz2:

1. Set **Fixed Frame** to `map`.

2. Click **Add**.
3. Select **By topic** → `/map` → **Map**.
4. Click **Add** again.
5. Select **By topic** → `/scan` → **LaserScan**.
6. You should see the map building in real time.

Save the Map

```
mkdir -p ~/maps
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab_map

# This creates:
# ~/maps/lab_map.pgm
# ~/maps/lab_map.yaml
```

NOTE

A good map has clean white open areas, solid black walls, and sharp corners. If the map looks noisy, drive more slowly and remap.

PHASE 10

Autonomous Navigation with Nav2

This phase makes the robot navigate autonomously to a goal point. The robot plans a path, drives to the goal, and avoids obstacles.

Install Nav2

```
sudo apt install -y ros-humble-navigation2 ros-humble-nav2-bringup
```

Create Nav2 Parameters File

```
mkdir -p ~/fyp_ws/src/qbot2_bringup/config
nano ~/fyp_ws/src/qbot2_bringup/config/nav2_params.yaml
```

Paste this basic configuration:

```
amcl:
  ros__parameters:
    use_sim_time: false
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    base_frame_id: base_footprint
    global_frame_id: map
    laser_model_type: likelihood_field
    max_beams: 60
    scan_topic: /scan

bt_navigator:
  ros__parameters:
    use_sim_time: false
    global_frame: map
    robot_base_frame: base_footprint

controller_server:
  ros__parameters:
    use_sim_time: false
    controller_frequency: 20.0
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
    min_theta_velocity_threshold: 0.001
```

```
local_costmap:
  local_costmap:
    ros__parameters:
      use_sim_time: false
      global_frame: odom
      robot_base_frame: base_footprint
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05

global_costmap:
  global_costmap:
    ros__parameters:
      use_sim_time: false
      global_frame: map
      robot_base_frame: base_footprint
      resolution: 0.05

planner_server:
  ros__parameters:
    use_sim_time: false
    planner_plugins: ['GridBased']
    GridBased:
      plugin: 'nav2_navfn_planner/NavfnPlanner'
      tolerance: 0.5
      use_astar: true
      allow_unknown: true
```

Build again:

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

Launch Full Navigation Stack

Use five SSH tabs.

Tab 1 – Kobuki Wheels

```
ros2 launch kobuki_ros kobuki_node-launch.py
```

Tab 2 – Kinect Sensor

```
ros2 launch openni2_launch openni2.launch.xml
```

Tab 3 – Depth to LaserScan

```
ros2 launch qbot2_bringup depth_to_scan.launch.py
```

Tab 4 – Robot State Publisher

```
ros2 launch qbot2_bringup robot_state.launch.py
```

Tab 5 – Nav2 with Saved Map

```
ros2 launch nav2_bringup bringup_launch.py \
  map:=/home/ubuntu/maps/lab_map.yaml \
  use_sim_time:=false \
  params_file:=/home/ubuntu/fyp_ws/src/qbot2_bringup/config/nav2_params.yaml
```

Wait until this appears:

```
# [lifecycle_manager] Managed nodes are active
```

Send Navigation Goals with RViz2

WINDOWS

Open a new X11 SSH tab and run:

```
ssh -X ubuntu@raspberrypi.local
rviz2
```

In RViz2:

1. Set **Fixed Frame** to `map`.
2. Add `/map` as a Map display.
3. Add `/scan` as a LaserScan display.
4. Add `/plan` as a Path display.
5. Add **TF**.

Set the robot's starting position:

1. Click **2D Pose Estimate**.
2. Click where the robot currently is on the map.
3. Drag to show the robot's direction.

Send a navigation goal:

1. Click **2D Nav Goal**.
2. Click the target position on the map.
3. Drag to set the target direction.
4. Watch the robot plan a path and move autonomously.

NOTE

This is the complete FYP demonstration. The robot uses the Kinect-built SLAM map to plan and execute autonomous navigation.

What is Happening Inside

Use this table to explain the system during supervisor meetings or in the final report.

Component	What it does	FYP relevance
Kinect RGB-D sensor	Captures depth images. Each pixel contains distance information.	Primary perception sensor.
depthimage_to_laserscan	Converts the depth image into a 2D laser scan.	Makes the Kinect compatible with standard SLAM and navigation packages.
RTAB-Map SLAM	Builds a map while estimating the robot position.	Core mapping and localisation algorithm.
Kobuki driver	Controls the wheel motors and reads odometry.	Provides robot mobility and motion feedback.
Robot State Publisher	Publishes coordinate frames using the URDF model.	Provides correct coordinate transformations.
AMCL	Localises the robot in the saved map using particles.	Helps the robot know where it is during navigation.
Nav2 Planner	Calculates a collision-free path to the goal.	Performs autonomous path planning.
Nav2 Controller	Sends velocity commands to follow the path.	Converts the path into robot motion.
Nav2 Costmap	Creates obstacle and safety zones.	Supports obstacle avoidance.
RViz2	Visualises the map, robot, laser scan, and planned path.	Used for debugging and demonstration.

Troubleshooting

Problem	Most likely cause	Solution
Cannot SSH into Raspberry Pi	Pi not booted or wrong IP address	Wait a few minutes after power on. Check the IP address from the router. Try `raspberrypi.local`.
Kinect not visible in `lsusb`	USB cable or power issue	Try another USB port. Check the cable. Reconnect the Kinect.
`/camera/depth` not publishing	OpenNI2 launch failed	Check `lsusb` first. Rerun udev rules. Try `sudo freenect-glview`.
`/scan` not showing data	depthimage_to_laserscan not running	Make sure Kinect topics are publishing first. Check topic names in the launch file.
`colcon build` fails	Missing dependencies	Run `rosdep install -from-paths src -ignore-src -r -y` and rebuild.
Map looks messy	Robot is moving too fast or Kinect is vibrating	Drive slower. Reduce speed. Mount the Kinect firmly.
SLAM loses tracking	Sharp or fast turns	Turn slowly. Restart SLAM with the `-d` flag and remap.
Nav2 waits for costmap	`/scan` is not reaching Nav2	Check `/scan`. Check frame names in URDF and depth-to-scan launch file.
Robot does not move to goal	Nav2 is not fully active	Wait for `Managed nodes are active`. Set 2D Pose Estimate before sending a goal.
RViz2 window not appearing	X11 forwarding not enabled	Use `ssh -X`. Check MobaXterm X11 settings.
Robot hits obstacles	Kinect field of view is limited	Drive slowly near obstacles. Increase costmap inflation radius if needed.

Problem	**Most likely cause**	**Solution**
Permission denied on `/dev/ttyUSB0`	USB permission not set	Run `sudo chmod 666 /dev/ttyUSB0`. Add the user to the `dialout` group and reboot.

Quick Reference Commands

Check What is Running

```
ros2 topic list
ros2 topic hz /camera/depth/image_raw
ros2 topic hz /scan
ros2 topic echo /odom
ros2 node list
ros2 doctor
```

Manual Robot Control

```
# Move forward:
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.2}, angular: {z: 0.0}}' --once

# Emergency stop:
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.0}, angular: {z: 0.0}}' --once
```

View Kinect Images

```
sudo apt install -y ros-humble-rqt-image-view

# In an X11 SSH tab:
ros2 run rqt_image_view rqt_image_view /camera/color/image_raw
ros2 run rqt_image_view rqt_image_view /camera/depth/image_raw
```

Save and Load Maps

```
# Save current map:
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab_map

# List saved maps:
ls ~/maps/
```

Rebuild Workspace

```
cd ~/fyp_ws
colcon build --symlink-install
source ~/.bashrc
```

14-Day Practical Plan

Day	What to do	You are done when
Day 1	Flash Ubuntu 22.04. Boot the Raspberry Pi. SSH from MobaXterm.	You see `ubuntu@raspberrypi` in the terminal.
Day 2	Update Ubuntu. Install ROS 2 Humble. Test talker and listener.	Hello World messages appear in the listener tab.
Day 3	Install Kobuki driver. Connect QBot 2. Test wheels.	`ros2 topic list` shows `/odom`.
Day 4	Install Kinect driver. Set USB permissions. Test Kinect.	`/camera/depth/image_raw` publishes around 30 Hz.
Day 5	Install depthimage_to_laserscan. Test `/scan`.	`ros2 topic echo /scan` shows range values.
Day 6	Create URDF. Set up robot state publisher. Build workspace.	`colcon build` finishes without errors.
Day 7	Install RTAB-Map. Launch all mapping nodes. Drive robot slowly.	RViz2 shows a rough map being built.
Day 8	Drive the full lab map. Save the map file.	`lab_map.pgm` and `lab_map.yaml` exist.
Day 9	Install Nav2. Configure parameters. Launch navigation stack.	`Managed nodes are active` appears.
Day 10	Set 2D Pose Estimate. Send first navigation goal.	Robot reaches a clicked goal point.
Day 11	Test several goal points and obstacle positions.	Robot reliably reaches different goals.
Day 12	Record video of SLAM and navigation demo.	Video clearly shows the full system working.
Day 13	Capture screenshots and write results.	Map and navigation screenshots are saved.
Day 14	Update final presentation.	Slides include system diagram, map, path planning, and demo results.

NOTE

Days 9 and 10 are the most important milestones. If the robot navigates autonomously using the Kinect-built map, the end-to-end system is complete.

Follow every phase in order.

Take it one step at a time and use this guide whenever you get stuck.